

# 第 07 章 | 集成學習：從單一模型到團隊預測

學生版公開講義 | 請搭配本課程 Colab，依照段落逐步操作與自我檢查。

來源說明：課程參考《Python Machine Learning, 3rd Edition》；原始範例程式碼採 MIT License。本檔由恩恩統計家教整理為學生版學習摘要，不含原書 PDF 或掃描內容。

## 1. 本章學習路線與實務價值

單一模型的泛化表現可能受抽樣結果與模型假設限制。集成學習 (Ensemble Learning) 透過組合多個模型，提高預測穩定性並降低部分估計誤差。

集成學習會組合多個分類器，利用投票、平均或加權方式產生最終預測。若成員模型具有足夠差異，組合結果通常比單一模型更穩定。

本章學習路線延續決策樹、SVM 與 KNN，本章使用投票、裝袋與提升方法，比較單一模型與組合模型的表現及穩定性。

學習目標清單

掌握多數決 (Voting) 的數學直覺，理解「獨立性」與「準確率 > 0.5」對機率權重分配的實務意義。

理解裝袋法 (Bagging) 如何透過「隨機重複抽樣」創造多樣性，藉此降低模型的變異 (Variance)。

剖析提升法 (Boosting) 如何針對錯誤樣本進行「加權特訓」，將弱學習器轉化為強分類器以降低偏差 (Bias)。

實踐 scikit-learn 集成模型 API 的開發流程，掌握 Pipeline 封裝與參數調優的高階技巧。

在深入複雜演算法前，我們必須先理解最直覺的協作方式：投票機制。

## 2. 核心觀念：組合多個模型

當基礎模型的錯誤不完全相關時，投票或平均可降低個別模型的偏差與偶然誤差。

白話觀念解析單一模型可能受抽樣或建模假設影響；組合多個彼此不同的模型，可降低個別模型誤差對最終預測的影響。

多數決 (Voting)：聽取多人意見，少數服從多數。就像一場陪審團投票。

裝袋法 (Bagging)：從訓練資料重複進行 bootstrap 抽樣，讓每個基礎模型看到略有不同的樣本，再彙整各模型的預測。

提升法 (Boosting)：針對錯題進行魔鬼特訓的家教模式。老師發現你第一題錯了，就在下一份考卷中「加大」第一題的權重與出現頻率，逼你學會為止。

重要術語中英對照表

| 中文術語 | 英文術語              | 核心功能摘要                |
|------|-------------------|-----------------------|
| 集成學習 | Ensemble Learning | 組合多個基底模型以提升預測效能的技術總稱。 |
| 元分類器 | Meta-classifier   | 組合多個基底分類器後所形成的高階分類框架。 |

| 中文術語 | 英文術語                       | 核心功能摘要                                |
|------|----------------------------|---------------------------------------|
| 泛化能力 | Generalization Performance | 模型對於未見過的「新資料」進行正確預測的能力。               |
| 裝袋法  | Bagging                    | 透過自助抽樣 (Bootstrap) 降低模型變異 (Variance)。 |
| 提升法  | Boosting                   | 透過序列式特訓降低模型偏差 (Bias)，將弱學習器轉強。         |
| 弱學習器 | Weak Learners              | 僅比隨機猜測好一點的模型，典型代表為「決策樹樁」。             |

專業筆記：在 Boosting 中，我們提到的「弱學習器」通常指決策樹樁 (Decision Stump)，也就是深度僅為 1、只有一個分裂節點的極簡決策樹。

### 3. 多數決 (Majority Voting)：群體決策的數學邏輯

投票機制不僅是直覺，更有嚴謹的統計學依據。但需注意一個前提：集體智慧發揮作用的前提是成員必須彼此獨立，且成員的正確率必須  $> 0.5$ 。

數學直覺計算說明根據二項式分佈原理，假設單一模型的錯誤率為  $\epsilon$ 。若我們找來 11 個彼此獨立且錯誤不相關的模型，當單一  $\epsilon=0.25$  時，集成後的錯誤率會大幅下降至 0.034。這證明了只要單一模型的錯誤率低於隨機猜測，錯誤率會隨著成員增加呈指數級下降。反之，若成員間缺乏獨立性（例如都用同樣的特徵處理、同樣的樣本訓練），他們會犯下同樣的錯誤，組合模型便難以降低誤差，甚至會導致「錯誤的共識」。

硬投票與軟投票的差異

硬投票 (Hard Voting)：傳統的多數決。在二元分類中稱為「Majority Voting」，在多類別設定中則稱為「相對多數決 (Plurality Voting)」，即選擇得票數最高的類別。

軟投票 (Soft Voting)：採取「機率加權」。如果模型 A 對預測「類別 0」有 90% 的把握，而模型 B 對預測「類別 1」只有 51% 的把握，軟投票會更重視把握度高的模型。在模型機率校準良好的情況下，軟投票通常比硬投票更精準。

重要公式導讀加權多數決的預測結果  $y^*$  可表示為： $y^* = \arg\max_i \sum_{j=1}^m w_j x_A(C_j(x)=i)$  其中  $x_A$  是指標函數，當基底分類器  $C_j$  的預測等於類別  $i$  時回傳 1； $w_j$  則是該模型的權重（例如巴菲特的票可以權重較大）。

### 4. Bagging 裝袋法：對抗過擬合的利器

Bagging (Bootstrap Aggregating) 的策略重點在於降低變異 (Variance)。當你面對容易過擬合的深層決策樹時，Bagging 是最佳處方。

機制分解

關鍵機制：取出後放回 (Sampling with replacement) 這又稱為 Bootstrap 抽樣。在每一輪 Bagging 中，我們隨機從訓練集中抽樣。因為抽完會放回，這會導致一個有趣的現象：某些樣本會出現多次，而某些樣本則完全不會出現。

多樣性來源：這種「透過排除產生的多樣性」是 Bagging 增加模型差異的來源。透過讓 500 棵樹各讀 500 份略有不同的考卷，模型群體就不會集體死背資料。

隨機森林 (Random Forest)：它是 Bagging 的進階特例。除了對「樣本」隨機抽樣，隨機森林還會對「特徵」進行隨機抽樣，進一步增強模型間的獨立性。

策略應用：Bagging 最適合搭配「高變異、低偏差」的模型（如未修剪的深層樹）。透過群體投票，它可以將原本生硬、帶有直角的決策邊界修剪得平滑圓潤，顯著提升模型的穩定性。

## 5. Boosting 提升法：精準打擊錯誤的特訓模式

Boosting 是一種序列式（Sequential），主要降低偏差（Bias），透過不斷針對錯誤樣本進行加重訓練，將「弱學習器」轉化為「強分類器」。

AdaBoost 四階段演化導讀

初始訓練：所有樣本權重相同。訓練第一個決策樹樁（Decision Stump）。

權重放大（Visual Insight）：在 AdaBoost 的演化圖中，你會發現某些資料點的尺寸變大了，這代表它們在上一輪被分錯，因此權重被調高。

針對性學習：下一個弱學習器為了降低整體的加權誤差，會被迫改變切線方向，優先去處理那些「體積變大」的困難樣本。

最終集成：將各輪產生的弱學習器依據其表現（ $\alpha$  係數）進行加權投票。

權重更新邏輯

計算錯誤率  $\epsilon$  → 算出模型重要性  $\alpha_j = 0.5 \log(\frac{1}{\epsilon} - 1)$ 。

更新樣本權重：猜對的權重縮小，猜錯的權重膨脹，交給下一個模型處理。

## 6. 程式碼流程導讀與 Colab 實作手冊

在 `scikit-learn` 中，複雜的集成理論被封裝得非常優雅。

核心實作流程分析

`BaggingClassifier`：適合處理深層樹。請善用 `n_jobs=-1`，這能利用多核心進行平行運算，大幅縮短訓練時間。

`AdaBoostClassifier`：預設基底模型為 `max_depth=1` 的決策樹樁。請注意，此處 `n_jobs` 參數無效或不適用，因為 Boosting 必須序列化執行（下一棵樹依賴上一棵的結果）。

Pipeline 整合與參數命名在使用 `GridSearchCV` 進行調參時，你會看到類似 `pipeline-1__clf__C` 的命名。

為什麼用雙底線 `__`？這是 `scikit-learn` 的巢狀參數存取邏輯。它代表「進入名為 `pipeline-1` 的步驟」→「尋找裡面的分類器 `clf`」→「設定參數 `C`」。

Colab 操作核取清單

- [] 環境庫匯入：包含 `sklearn.ensemble` 與 `sklearn.pipeline`。
- [] 資料集載入：使用 Wine 資料集。

\_專業提示：\_ 若 UCI 伺服器不穩導致 `pd.read_csv(URL)` 失敗，請將 URL 字串替換為本地路徑字串（如 `./wine.data`）並上傳檔案。

- [] 模型封裝：將 `StandardScaler` 與分類器包入 Pipeline。
- [] 效能監控：觀察 Bagging (平行) 與 AdaBoost (序列) 的執行時間差異。

## 7. 常見錯誤排除與本章總結

集成學習雖然強大，但請記住「天下沒有白吃的午餐（No Free Lunch）」。

### 疑難排解 FAQ

為什麼在 Colab 跑 500 棵樹會卡死？運算複雜度問題。特別是 Boosting 無法平行運算，建議學習示範時先將 `n_estimators` 設為 10 觀察行為，再逐步增加。

為什麼對笨模型（高偏差）用 Bagging 沒效？Bagging 是為了治療「過擬合（高變異）」。如果模型本來就「欠擬合（太笨）」，你應該改用 Boosting。

實務上的取舍：強大的模型往往伴隨龐大的工程成本。以 Netflix 100 萬美金大獎 為例，奪冠的集成模型雖然準確，但 Netflix 最後並未全面採用。誠如官方所言：「離線評估的微小準確率提升，無法抵銷將其匯入生產環境所需的龐大工程研發與運算成本。」

### 實踐任務與自評

任務：嘗試調整 `n_estimators`，觀察決策邊界如何從生硬的直角（單一樹）演變為平滑的曲線（集成後）。

核心自評表：

| 核心問題                              | 自測重點                     |
|-----------------------------------|--------------------------|
| 1. 多數決發揮作用的兩大前提是什麼？               | 獨立性、個別準確率 > 0.5          |
| 2. Bagging 抽樣後，是否所有樣本都會被訓練到？      | 否，部分樣本會因隨機性被排除，這創造了多樣性   |
| 3. AdaBoost 圖示中點的大小代表什麼？          | 樣本權重（Weight）             |
| 4. 為何 AdaBoost 訓練速度通常比 Bagging 慢？ | 因為它是序列執行，無法平行運算          |
| 5. 為什麼 Netflix 不採用獲勝的集成模型？        | 運算成本與工程維護代價過高，效能增益不具經濟效益 |

本章總結：集成學習透過模型差異與預測彙整，降低單一模型的不穩定性。完成本章後，你應能說明 Voting、Bagging、Boosting 與隨機森林的差異及適用情境。